APPENDIX A

CODE SNIPPETS

Appendix A. Code Snippets

This appendix presents selected source code of Project TIMBANG. The snippets were chosen because they implement the core of the system: how a daily price and its confidence label are computed, how data and roles are protected, how prices are submitted and imported, and how the province-wide price bulletin is produced. The mobile application and website are written in Dart (Flutter); the server logic is written in TypeScript (Cloud Functions for Firebase); access control is written in the Firestore Security Rules language. The complete source code is available in the project repository (github.com/MysteriousAlarm/Timbang).

Snippet A.1. Median price, outlier handling and confidence label

Source file: `functions/src/confidence.ts`

The price shown on the board is the median of the last 24 hours of reports, never the average or the latest single report. Reports more than 30% away from the median are set aside as outliers (official prices are never set aside), and the confidence label is assigned from the sources that remain.

```
export function median(values: number[]): number {
  if (values.length === 0) throw new Error('median of empty list');
  const s = [...values].sort((a, b) => a - b);
  const mid = Math.floor(s.length / 2);
  const m = s.length % 2 ? s[mid] : (s[mid - 1] + s[mid]) / 2;
  return Math.round(m * 100) / 100;
}

const deviation = (price: number, m: number) => Math.abs(price - m) / m;

/**
 * Splits reports into valid and outlier around the median.
 * - Rejected reports are ignored entirely (an admin decision).
 * - Official reports are never outliers: they are the reference.
 * - If every non-official report would be an outlier, none are marked
 *   (e.g. two disagreeing reports): there is no majority to trust.
 */
export function markOutliers(reports: CalcReport[]): {
  valid: CalcReport[];
  outliers: CalcReport[];
  median: number;
} {
  const candidates = reports.filter((r) => r.status !== 'rejected');
  if (candidates.length === 0) return { valid: [], outliers: [], median: NaN };
  const first = median(candidates.map((r) => r.price));
  const outliers = candidates.filter(
    (r) => r.source !== 'official' && deviation(r.price, first) > OUTLIER_THRESHOLD,
  );
  if (outliers.length === candidates.length) {
    return { valid: candidates, outliers: [], median: first };
  }
  const valid = candidates.filter((r) => !outliers.includes(r));
  return { valid, outliers, median: median(valid.map((r) => r.price)) };
}

export function classifyConfidence(valid: CalcReport[], m: number): Confidence {
```

```
  if (valid.some((r) => r.source === 'official')) return 'high';
  const agreeing = valid.filter((r) => deviation(r.price, m) <= AGREEMENT_THRESHOLD);
  if (agreeing.length >= 3 && agreeing.some((r) => r.source === 'reporter')) return
'high';
  if (valid.some((r) => r.source === 'reporter')) return 'medium';
  return 'low';
}
```

Snippet A.2. Deny-by-default security rules and self sign-up validation

Source file: `firestore.rules`

Roles are read only from verified custom claims on the login token, never from data the app writes. A new user may only create their own profile as a farmer or buyer, with the phone number that was verified by SMS, and only the fields allowed under data minimization (RA 10173).

```
rules_version = '2';

// Project TIMBANG Firestore rules.
// Deny by default. Roles come ONLY from custom claims (request.auth.token.role),
// which are set by the admin-only `setUserRole` Cloud Function.
// The `role` field in users/{uid} is a display mirror and is never trusted here.
service cloud.firestore {
  match /databases/{database}/documents {

    // ---------- helpers ----------
    function signedIn() {
      return request.auth != null;
    }
    function claimRole() {
      return signedIn() && ('role' in request.auth.token) ? request.auth.token.role :
null;
    }
    function isAdmin() {
      return claimRole() == 'admin';
    }
    function isOwner(uid) {
      return signedIn() && request.auth.uid == uid;
    }

    // The 15 municipalities of Nueva Vizcaya (keep in sync with app core/config and
functions config.ts).
    function municipalities() {
      return ['Alfonso Castañeda', 'Ambaguio', 'Aritao', 'Bagabag', 'Bambang',
              'Bayombong', 'Diadi', 'Dupax del Norte', 'Dupax del Sur', 'Kasibu',
              'Kayapa', 'Quezon', 'Santa Fe', 'Solano', 'Villaverde'];
    }
    function validName(v) {
      return v is string && v.trim().size() >= 2 && v.size() <= 80;
    }
    function validMunicipality(v) {
      return v is string && v in municipalities();
    }
    function validFcmTokens(v) {
      return v is list && v.size() <= 10;
    }

    // ---------- users ----------
    // Data minimization (RA 10173): only name, phone, municipality are personal data.
    match /users/{uid} {
      allow read: if isOwner(uid) || isAdmin();

      // Self-signup: only farmer or buyer, unverified, active. Phone must match the
      // verified phone on the auth token, so nobody can claim another number.
      allow create: if isOwner(uid)
        && request.resource.data.keys().hasOnly(
```

```
             ['name', 'phone', 'municipality', 'role', 'verified', 'status',
              'createdAt', 'consentAt', 'fcmTokens'])
        && request.resource.data.keys().hasAll(
             ['name', 'phone', 'municipality', 'role', 'verified', 'status',
              'createdAt', 'consentAt'])
        && validName(request.resource.data.name)
        && request.resource.data.phone is string
        && request.resource.data.phone == request.auth.token.phone_number
        && validMunicipality(request.resource.data.municipality)
        && request.resource.data.role in ['farmer', 'buyer']
        && request.resource.data.verified == false
        && request.resource.data.status == 'active'
        && request.resource.data.createdAt == request.time
        && request.resource.data.consentAt == request.time
        && (!('fcmTokens' in request.resource.data) ||
validFcmTokens(request.resource.data.fcmTokens));
```

Snippet A.3. Admin-only role assignment with audit logging

Source file: `functions/src/roles.ts`

Only an administrator can grant a role. Every change is written to the audit log, and an administrator cannot accidentally remove their own admin role.

```
/**
 * Admin-only: set a user's role custom claim.
 * Clients can never set their own role; this is the only path that writes the claim.
 */
export async function setUserRoleHandler(
  caller: Caller | undefined,
  data: unknown,
  deps: RoleDeps,
): Promise<{ uid: string; role: Role }> {
  if (!caller) throw new HttpsError('unauthenticated', 'Sign in first.');
  if (caller.role !== 'admin') throw new HttpsError('permission-denied', 'Admins
only.');

  const { uid, role } = parseSetUserRoleInput(data);
  if (uid === caller.uid && role !== 'admin') {
    // Prevents an admin from locking the province out by accident.
    throw new HttpsError('failed-precondition', 'Admins cannot remove their own admin
role.');
  }

  const existing = await deps.getClaims(uid);
  if (existing === null) throw new HttpsError('not-found', 'User not found.');

  const previous = typeof existing.role === 'string' ? existing.role : 'none';
  await deps.setClaims(uid, { ...existing, role });
  // reporter/admin are vetted by an admin, so they are marked verified.
  await deps.mirrorRole(uid, role, role === 'reporter' || role === 'admin');
  await deps.audit({
    actorId: caller.uid,
    action: 'set_user_role',
    targetPath: `users/${uid}`,
    detail: `${previous} -> ${role}`,
  });
  return { uid, role };
}
```

Snippet A.4. Server-side price report validation and rate limiting

Source file: `functions/src/priceReports.ts`

The trust tier of a report (reporter or community) is decided by the server from the user's role, so it cannot be faked by the app. Each user is limited to 20 reports per hour, and repeated reports of the same item and market within 10 minutes are refused.

```
/** Source tier comes from the verified role claim, never from the client. */
export function sourceFor(role: unknown): Source {
  return role === 'reporter' || role === 'admin' ? 'reporter' : 'community';
}

/** Any signed-in user may report; reporters/admins count as verified, others as
community. */
export async function submitPriceReportHandler(
  caller: Caller,
  data: unknown,
  store: PriceStore,
): Promise<{ id: string; source: Source }> {
  if (!caller) throw new HttpsError('unauthenticated', 'Sign in first.');
  const d = (typeof data === 'object' && data !== null ? data : {}) as Record<string,
unknown>;
  const extra = Object.keys(d).filter((k) => !['commodityId', 'marketId',
'price'].includes(k));
  if (extra.length) throw new HttpsError('invalid-argument', 'Unexpected fields.');
  const { commodityId, marketId, price } = d;
  if (typeof commodityId !== 'string' || !ID.test(commodityId)) {
    throw new HttpsError('invalid-argument', 'Invalid commodityId.');
  }
  if (typeof marketId !== 'string' || !ID.test(marketId)) {
    throw new HttpsError('invalid-argument', 'Invalid marketId.');
  }
  if (!validPrice(price)) {
    throw new HttpsError(
      'invalid-argument',
      `price must be ${PRICE_MIN}-${PRICE_MAX}, 2 decimals.`,
    );
  }

  const user = await store.getUser(caller.uid);
  if (!user) throw new HttpsError('failed-precondition', 'Create your profile first.');
  if (user.status !== 'active') throw new HttpsError('permission-denied', 'Account
suspended.');

  const [commodity, market] = await Promise.all([
    store.getCommodity(commodityId),
    store.getMarket(marketId),
  ]);
  if (!commodity?.active) throw new HttpsError('not-found', 'Unknown or inactive
commodity.');
  if (!market?.active) throw new HttpsError('not-found', 'Unknown or inactive
market.');

  const source = sourceFor(caller.role);
  const assigned = user.assignedMarketIds ?? [];
  if (caller.role === 'reporter' && assigned.length > 0 && !
assigned.includes(marketId)) {
    throw new HttpsError('permission-denied', 'You are not assigned to this market.');
  }

  // Rate limits (Section 10): per hour, and no repeats of the same pair within a few
minutes.
  const now = store.now();
  const recent = await store.reportsByUserSince(caller.uid, now - 60 * 60 * 1000);
  if (recent.length >= REPORTS_PER_HOUR) {
    throw new HttpsError('resource-exhausted', 'Too many reports. Try again later.');
  }
  const dupSince = now - DUPLICATE_REPORT_MINUTES * 60 * 1000;
```

```
  if (
    recent.some(
      (r) => r.commodityId === commodityId && r.marketId === marketId && r.timestamp >
dupSince,
    )
  ) {
    throw new HttpsError('already-exists', 'You just reported this. Try again later.');
  }

  const [id] = await store.addReports([
    {
      commodityId,
      marketId,
      price,
      unit: commodity.unit,
      reporterId: caller.uid,
      reporterRole: typeof caller.role === 'string' ? caller.role : 'member',
      source,
      timestamp: now,
      status: 'valid',
      isDemo: false,
    },
  ]);
  return { id, source };
}
```

Snippet A.5. SMS webhook signature verification and number masking

Source file: `functions/src/sms/gateway.ts`

Incoming SMS messages are accepted only when their HMAC-SHA256 signature is valid, checked in constant time. Phone numbers are masked before anything is written to the logs.

```
export function hmacHex(secret: string, body: Buffer | string): string {
  return createHmac('sha256', secret).update(body).digest('hex');
}

/** Constant-time HMAC check of `x-timbang-signature` over the raw body. Fails closed.
*/
export function verifyHmac(secret: string, body: Buffer | string, signature: unknown):
boolean {
  if (!secret || typeof signature !== 'string' || !/^[0-9a-f]{64}$/i.test(signature))
return false;
  const expected = Buffer.from(hmacHex(secret, body), 'hex');
  const got = Buffer.from(signature, 'hex');
  return expected.length === got.length && timingSafeEqual(expected, got);
}

/** "+639171234567" -> "+63917***4567" for logs. Never log full numbers. */
export function maskNumber(n: string): string {
  return n.length > 7 ? `${n.slice(0, 6)}***${n.slice(-4)}` : '***';
}
```

Snippet A.6. Province-wide price summary

Source file: `app/lib/features/price_board/application/board_providers.dart`

The home screen shows the whole province by default. Each vegetable is summarised across all markets: the median of the markets' medians, the price range, and the lowest confidence among those markets, so the province view never overstates how much a price can be trusted.

```
/// Summarises each commodity across active markets:
/// - price: the median of the markets' medians (one market cannot pull it alone);
```

```
/// - confidence: the LOWEST among those markets (never overstates trust);
/// - age: the newest report; counts and source tiers: summed;
/// - change: median of today vs median of yesterday where known (±3% = steady).
/// Only entries in the commodity's most common unit are combined, so kg and bundle
never mix.
List<ProvinceRow> provinceRows(
  List<BoardEntry> entries,
  List<Commodity> commodities,
  List<Market> markets,
) {
  final activeMarkets = {
    for (final m in markets)
      if (m.active) m.id,
  };
  final byCommodity = <String, List<BoardEntry>>{};
  for (final e in entries) {
    if (activeMarkets.contains(e.marketId)) {
      byCommodity.putIfAbsent(e.commodityId, () => []).add(e);
    }
  }
  final rows = <ProvinceRow>[];
  for (final c in commodities.where((c) => c.active)) {
    final list = byCommodity[c.id];
    if (list == null || list.isEmpty) continue;
    final unitCount = <String, int>{};
    for (final e in list) {
      unitCount[e.unit] = (unitCount[e.unit] ?? 0) + 1;
    }
    final unit = unitCount.entries.reduce((a, b) => a.value >= b.value ? a : b).key;
    final same = list.where((e) => e.unit == unit).toList();
    final prices = [for (final e in same) e.medianPrice];
    final median = _median(prices);
    final prevs = [for (final e in same) ?e.previousMedian];
    final prev = prevs.isEmpty ? null : _median(prevs);
    final trend = prev == null || prev == 0
        ? Trend.flat
        : (median - prev) / prev > 0.03
        ? Trend.up
        : (median - prev) / prev < -0.03
        ? Trend.down
        : Trend.flat;
    final lowest = same.map((e) => e.confidence).reduce((a, b) => a.index >= b.index ?
a : b);
    rows.add((
      commodity: c,
      markets: same.length,
      low: prices.reduce((a, b) => a < b ? a : b),
      high: prices.reduce((a, b) => a > b ? a : b),
      entry: BoardEntry(
        commodityId: c.id,
        marketId: '',
        unit: unit,
        medianPrice: median,
        confidence: lowest,
        reportCount: same.fold(0, (n, e) => n + e.reportCount),
        trend: trend,
        latestReportAt: same.map((e) => e.latestReportAt).reduce((a, b) => a.isAfter(b)
? a : b),
        previousMedian: prev,
        official: same.fold(0, (n, e) => n + e.official),
        reporter: same.fold(0, (n, e) => n + e.reporter),
        community: same.fold(0, (n, e) => n + e.community),
        isDemo: same.any((e) => e.isDemo),
      ),
    ));
  }
  rows.sort(
    (a, b) =>
a.commodity.nameFil.toLowerCase().compareTo(b.commodity.nameFil.toLowerCase()),
  );
```

```
  return rows;
}
```

Snippet A.7. Reading official price sheets as the office prepares them

Source file: `app/lib/features/reports/domain/official_csv.dart`

Administrators can import the Excel or CSV file from the provincial agriculture office without reformatting it: vegetables and markets may be written by name, title rows are skipped, and both the "one row per price" and the "one column per market" layouts are accepted.

```
/// Reads an official price sheet as it comes from the office, in either layout:
/// - long: one row per price, with commodity, market and price columns (date
optional);
/// - wide: one row per commodity and one column per market, each cell a price.
/// Commodities and markets may be written as their catalog ID or their name
/// ([commodityNames] / [marketNames] map [normKey] of a name to an ID).
/// Title rows above the header and sub-heading rows without prices are skipped.
OfficialImport parseOfficialTable(
  List<List<String>> table, {
  required Set<String> commodityIds,
  required Set<String> marketIds,
  Map<String, String> commodityNames = const {},
  Map<String, String> marketNames = const {},
}) {
  if (table.isEmpty) return const OfficialImport([]);
  String? resolve(String raw, Set<String> ids, Map<String, String> names) {
    final v = raw.trim().toLowerCase();
    if (ids.contains(v)) return v;
    final byName = names[normKey(raw)];
    return byName != null && ids.contains(byName) ? byName : null;
  }

  int? findCol(List<String> keys, List<String> names) {
    final i = keys.indexWhere(names.contains);
    return i < 0 ? null : i;
  }

  bool looksLikeHeader(List<String> keys) =>
      findCol(keys, _aliases['commodityId']!) != null &&
      (keys.any(_isPrice) || keys.any((k) => resolve(k, marketIds, marketNames) !=
null));

  final h = findHeaderRow(table, looksLikeHeader);
  final keys = [for (final c in table[h]) normKey(c)];
  final cCol = findCol(keys, _aliases['commodityId']!);
  final mCol = findCol(keys, _aliases['marketId']!);
  final dCol = findCol(keys, _aliases['date']!);
  final pIdx = keys.indexWhere(_isPrice);
  final pCol = pIdx < 0 ? null : pIdx;
  // Wide layout: headers that are markets.
  final marketCols = <int, String>{
    for (var i = 0; i < keys.length; i++)
      if (i != cCol && i != dCol && resolve(table[h][i], marketIds, marketNames) !=
null)
        i: resolve(table[h][i], marketIds, marketNames)!,
  };
  final wide = mCol == null && marketCols.isNotEmpty;
  final missing = [
    if (cCol == null) 'commodityId',
    if (!wide && mCol == null) 'marketId',
    if (!wide && pCol == null) 'price',
  ];
  if (missing.isNotEmpty) return OfficialImport(const [], headerIssues: missing);
```

Snippet A.8. Remembering the chosen municipality

Source file: `app/lib/features/price_board/application/price_scope.dart`

The place selected in the header (the whole province or one of the 15 municipalities) is saved on the device so the app opens to the same place the next time.

```
class PriceScopeController extends Notifier<String?> {
  @override
  String? build() {
    final saved = ref.watch(savedPriceScopeProvider);
    return AppConfig.municipalities.contains(saved) ? saved : null;
  }

  /// null = all of Nueva Vizcaya.
  Future<void> choose(String? municipality) async {
    state = AppConfig.municipalities.contains(municipality) ? municipality : null;
    try {
      final prefs = await SharedPreferences.getInstance();
      await prefs.setString(_prefsKey, state ?? '');
    } catch (_) {
      // Not saved (private browser window): the choice still applies for this session.
    }
  }
}

final priceScopeProvider = NotifierProvider<PriceScopeController, String?>(
  PriceScopeController.new,
);
```

APPENDIX B

APPROVED LETTER

Appendix B. Approved Letter

The signed and approved letter is attached on the following page.

Insert the scanned copy of the signed and approved letter here.

(In Word: delete this box, then Insert > Pictures and set the width to 6 inches.)

APPENDIX C

COPY OF ASSESSMENT FORM

PROJECT TIMBANG

Usability Assessment Form (System Usability Scale)

Talatanungan sa Paggamit ng Sistema

Instructions / Panuto: This form is answered after you try TIMBANG. Your answers are used only for this study and are kept confidential under the Data Privacy Act of 2012 (RA 10173). You may stop at any time. / Sagutan ito pagkatapos subukan ang TIMBANG. Gagamitin lamang ang iyong sagot para sa pag-aaral na ito at mananatiling kumpidensiyal.

Part I. Respondent Profile / Impormasyon ng Sumasagot

Name (optional) / Pangalan:	Date / Petsa:
Municipality / Bayan:	Age / Edad: ☐ below 25 ☐ 25–40 ☐ 41–60 ☐ above 60
Role / Tungkulin: ☐ Farmer / Magsasaka ☐ Buyer / Mamimili ☐ Agriculture staff	Phone used / Telepono: ☐ Smartphone ☐ Basic phone
Times you sell or buy produce per week:	Evaluator / Tagasuri:

Part II. Test Tasks / Mga Gawain (to be filled in by the evaluator)

No.	Task / Gawain	Done without help	Done with help	Not done	Time (min)
1	Log in with your mobile number. / Mag-log in gamit ang iyong numero.	☐	☐	☐	
2	Find today's price of tomato (kamatis). How old is it? / Hanapin ang presyo ng kamatis ngayon. Gaano ito katagal na?	☐	☐	☐	
3	Switch to your municipality and compare the price with other markets. / Pumili ng iyong bayan at ihambing ang presyo sa ibang palengke.	☐	☐	☐	
4	Farmer: post your produce. Buyer: send a question about a listing. / Magsasaka: mag-post ng ani. Mamimili: magtanong tungkol sa isang ani.	☐	☐	☐	
5	Set a price alert. / Mag-set ng alerto sa presyo.	☐	☐	☐	
6	Basic phone: text PRESYO KAMATIS BAYOMBONG. / I-text ang PRESYO KAMATIS BAYOMBONG.	☐	☐	☐	

Part III. System Usability Scale / Antas ng Kadalian ng Paggamit

Put a check (✓) on the number that shows how much you agree. / Lagyan ng tsek (✓) ang numerong tumutugma sa iyong pagsang-ayon. **1** – Strongly disagree / Lubos na hindi sang-ayon **2** – Disagree / Hindi sang-ayon **3** – Neutral / Walang kinikilingan **4** – Agree / Sang-ayon **5** – Strongly agree / Lubos na sang-ayon

No.	Statement / Pahayag	1	2	3	4	5
1	I would like to use TIMBANG frequently. Gusto kong gamitin ang TIMBANG nang madalas.	☐	☐	☐	☐	☐
2	I found TIMBANG unnecessarily complex. Masyadong komplikado ang TIMBANG.	☐	☐	☐	☐	☐
3	I thought TIMBANG was easy to use. Madaling gamitin ang TIMBANG.	☐	☐	☐	☐	☐
4	I would need the help of a technical person to be able to use TIMBANG. Kakailanganin ko ng tulong ng ibang tao para magamit ito.	☐	☐	☐	☐	☐
5	I found the various functions in TIMBANG were well integrated. Magkakaugnay nang maayos ang mga bahagi nito.	☐	☐	☐	☐	☐
6	I thought there was too much inconsistency in TIMBANG. Masyadong maraming hindi pagkakatugma sa app.	☐	☐	☐	☐	☐
7	I would imagine that most people would learn to use TIMBANG very quickly. Mabilis itong matututunan ng karamihan.	☐	☐	☐	☐	☐
8	I found TIMBANG very cumbersome to use. Mabigat o mahirap itong gamitin.	☐	☐	☐	☐	☐
9	I felt very confident using TIMBANG. Kumpiyansa ako habang ginagamit ito.	☐	☐	☐	☐	☐
10	I needed to learn a lot of things before I could get going with TIMBANG. Marami akong kailangang matutunan bago ito magamit.	☐	☐	☐	☐	☐

Part IV. Comments and Suggestions / Puna at Mungkahi

What did you like most? / Ano ang pinakanagustuhan mo?

_____

_____

What was difficult or confusing? / Ano ang mahirap o nakalilito?

_____

_____

What should be added or changed? / Ano ang dapat idagdag o baguhin?

_____

_____

_____ _____

Signature of respondent / Lagda ng sumasagot Signature of evaluator / Lagda ng tagasuri

Scoring Guide (for the researcher)

The System Usability Scale (Brooke, 1996) gives one score from 0 to 100 per respondent. Compute it as follows:

1. For each odd-numbered item (1, 3, 5, 7, 9): subtract 1 from the answer.
2. For each even-numbered item (2, 4, 6, 8, 10): subtract the answer from 5.
3. Add the ten results and multiply the sum by 2.5.

SUS score	Interpretation
above 80.3	Excellent (grade A)
68 to 80.3	Good, above average
68	Average
51 to 67	Below average, needs improvement
below 51	Poor

Report the mean, median and range of the SUS scores per group (farmers, buyers, agriculture staff), the task completion rate (done without help ÷ respondents), the average time per task, and a summary of the comments. The target for the pilot is a mean SUS score of at least 70.

Reference: Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.

APPENDIX D

COPY OF SYSTEM EVALUATION FORM

Appendix D. Copy of System Evaluation Form

The system evaluation form used in this study is attached on the following page.

Insert the copy of the system evaluation form here.

(In Word: delete this box, then Insert > Pictures, or paste the form's pages.)

APPENDIX E

DOCUMENTATION

Appendix E. Documentation

This appendix contains the user manual of Project TIMBANG (E.1), its technical documentation (E.2), and the photo documentation of the study (E.3). The screenshots were taken from the first build (version 0.1.1) using demonstration data, which the application labels as "DEMO"; the prices shown are not real prices.

E.1 User Manual

About TIMBANG

TIMBANG ("to weigh") is a daily vegetable price board and produce marketplace for the farmers and buyers of Nueva Vizcaya. It shows the typical (median) price of each vegetable per market, how much that price can be trusted, and when it was last updated. Farmers can post what they are harvesting and buyers can contact them directly. TIMBANG runs as an Android application, as a website, and through SMS for basic phones. It only reports observed prices and never sets prices.

User	What they can do
Farmer (Magsasaka)	Check and compare prices, post produce, answer buyer inquiries, set price alerts, report prices
Buyer (Mamimili)	Check and compare prices, browse produce, send inquiries to farmers, set price alerts
Price reporter (Tagapag-ulat)	Submit price reports that carry more weight; approved by an administrator
Administrator	Manage the vegetable and market lists, import official prices, manage users, moderate content, view pilot metrics

1. Getting started

1.1 Choose a language. On first opening, choose English or Filipino. The choice can be changed later in Profile.

1.2 Log in with your mobile number. Enter your number (e.g. 0917 123 4567) and tap **Send code**. A 6-digit code is sent by SMS. Enter it and tap **Verify**. No password is needed.

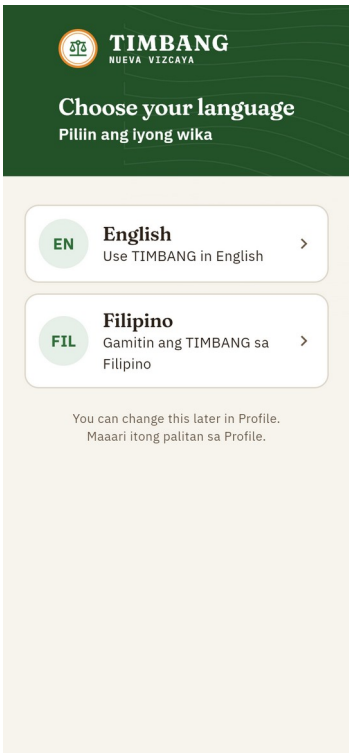


Figure E-1. Language selection

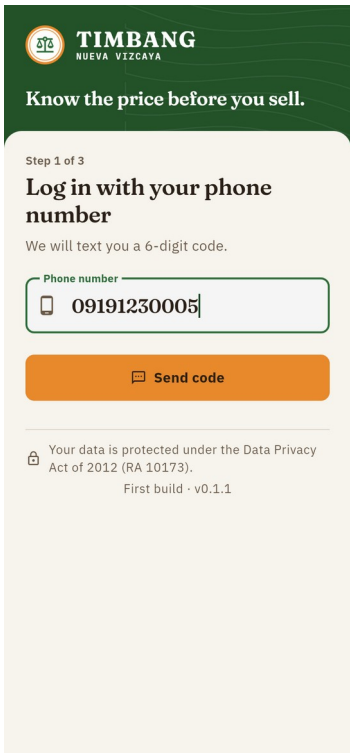


Figure E-2. Phone number login

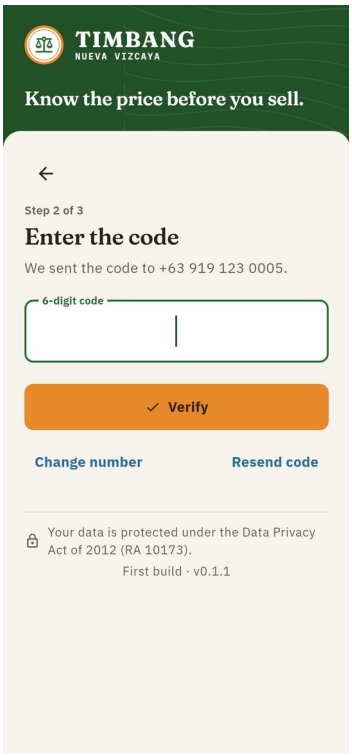


Figure E-3. Entering the SMS code

1.3 Create your profile. New users enter their full name, choose their municipality, and choose whether they are a farmer or a buyer. Reporter and administrator roles are given only by an administrator.

1.4 Read and agree to the Privacy Notice. Tap **Read and agree**. The Data Privacy Act notice opens on its own screen. The checkbox at the bottom becomes available only after scrolling to the end; tick it and tap **I agree**, then tap **Save profile**.

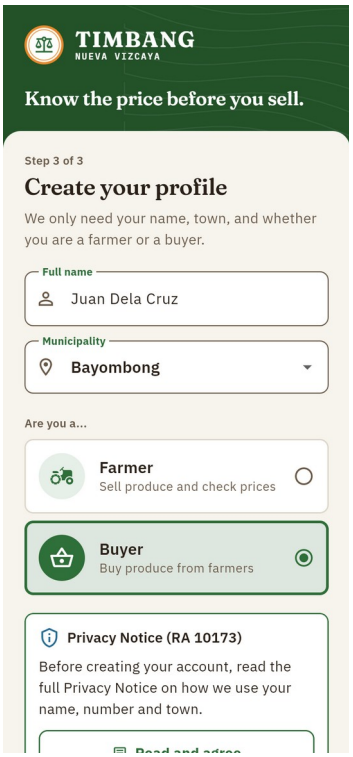


Figure E-4. Profile setup

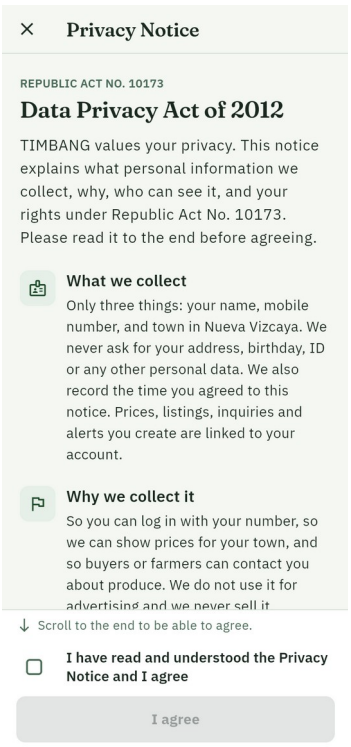


Figure E-5. Privacy Notice (top)

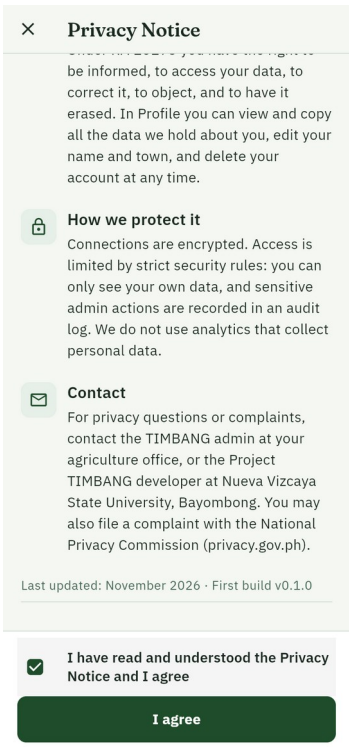


Figure E-6. Agreeing at the end of the notice

2. Checking prices

2.1 The daily price bulletin. The home screen (**Prices** tab) shows every vegetable across all markets in Nueva Vizcaya, grouped by category. Each row shows the median price per unit, the change since yesterday, the trust meter, the price range across markets, the number of reports and when it was last updated. A price with no report in 24 hours is still shown, marked **Stale**.

Trust meter	Meaning
▮▮▮ High	An official price (DA, DAR, NVAT), or three or more agreeing reports including a verified reporter
▮▮▯ Medium	Reports from verified price reporters
▮▯▯ Low	Community reports only, until they are confirmed

2.2 Choose a place. Tap **Prices in · All of Nueva Vizcaya** at the top and pick one of the 15 municipalities to see the prices at its market(s). The choice is remembered on the phone, and the produce listings follow the same town.

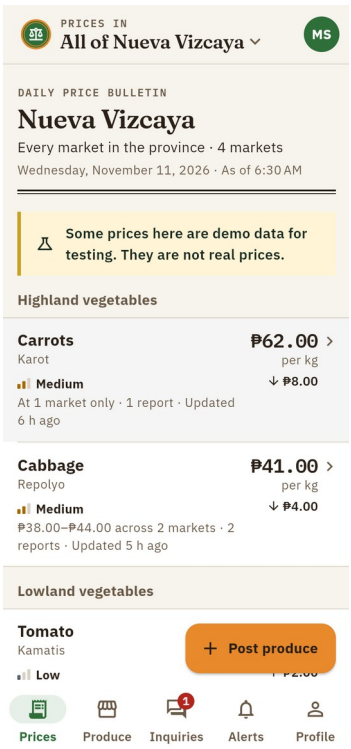


Figure E-7. Province-wide price bulletin

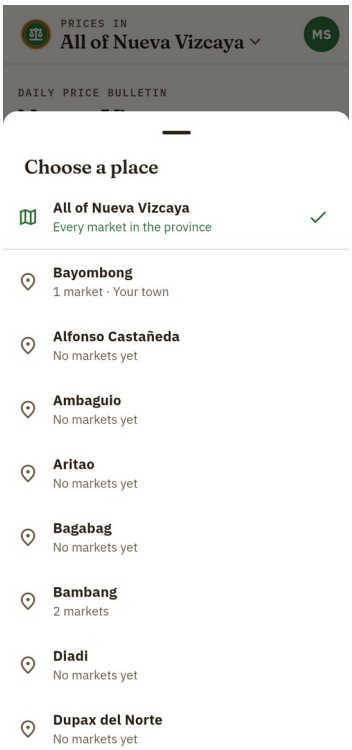


Figure E-8. Choosing a municipality

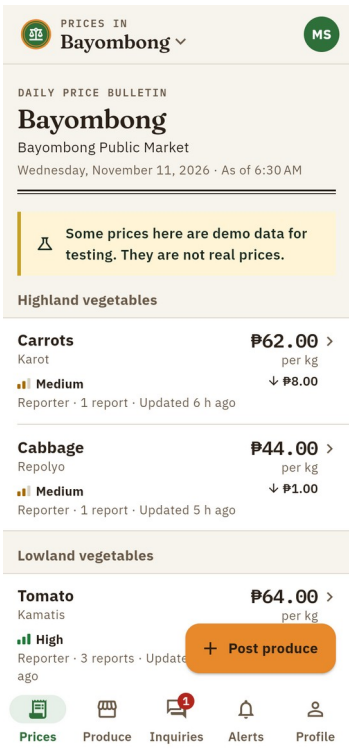


Figure E-9. Prices in one town

2.3 Compare markets. Tap a vegetable to see its price at your market, yesterday's price, and its price at every other market, with your town first and the cheapest market marked.

2.4 Report a price. Under **Services**, tap **Report a price**, choose the vegetable and market, enter the price you saw, and tap **Send report**. Community reports count as low confidence until a verified report matches them.

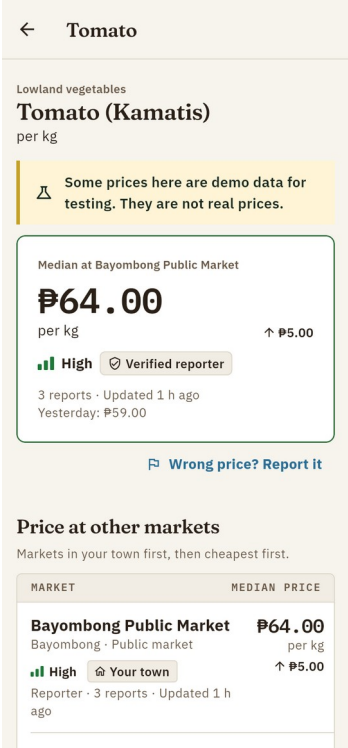


Figure E-10. Price of one vegetable

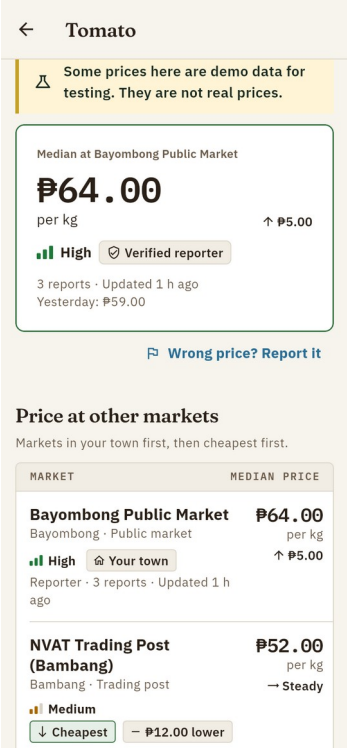


Figure E-11. Comparison across markets

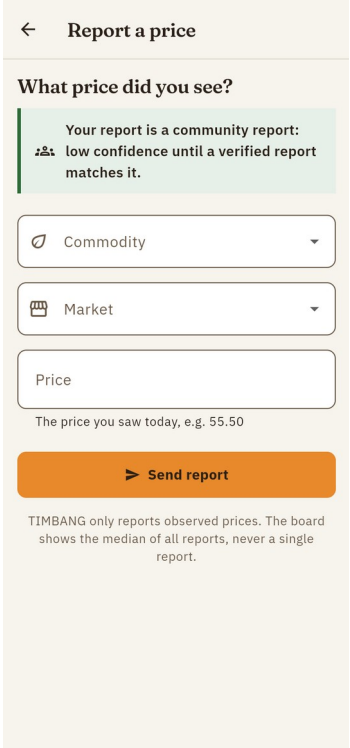


Figure E-12. Reporting a price

3. Produce listings

3.1 Browse produce. The **Produce** tab lists what farmers are selling, filtered by vegetable and municipality and sorted by newest or cheapest.

3.2 Post produce (farmers). Tap **Post produce**, choose the vegetable, enter the quantity, asking price, municipality and harvest date, optionally add a note and up to three photos, and tap **Post produce**. A listing stays active for 14 days and can be marked sold or renewed.

3.3 Contact the farmer (buyers). Open a listing to see its details, how its price compares with the market median, and the farmer's number. Call, text, or send one inquiry through the app.

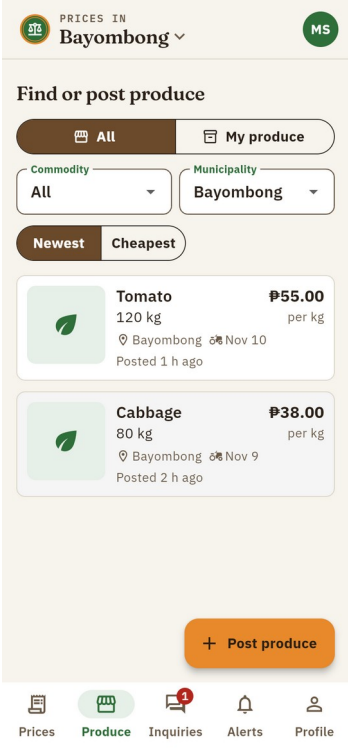


Figure E-13. Produce listings

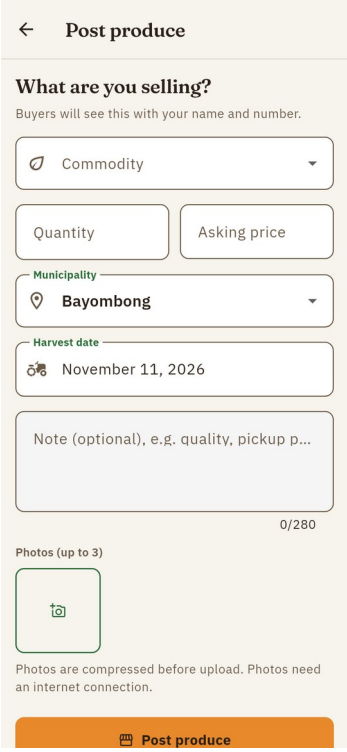


Figure E-14. Posting produce

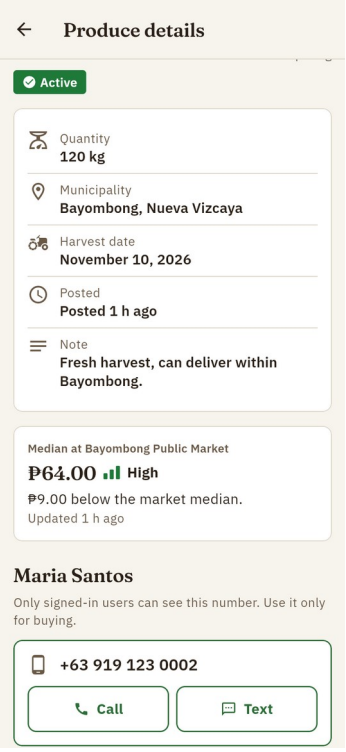


Figure E-15. Listing details and contact

4. Inquiries, alerts and profile

4.1 Inquiries. Farmers see buyers' messages in the **Inquiries** tab, with buttons to call or text back, mark as read, or close. A red badge shows new inquiries.

4.2 Price alerts. In **Alerts**, tap **Add alert**, choose a vegetable and market, choose **Rises to** or **Falls to**, and enter the price. A notification is sent once each time the median crosses that price.

4.3 Profile. Profile shows your name, number and municipality, lets you change the language, read the Privacy Notice again, view or copy all your data, sign out, or delete your account.

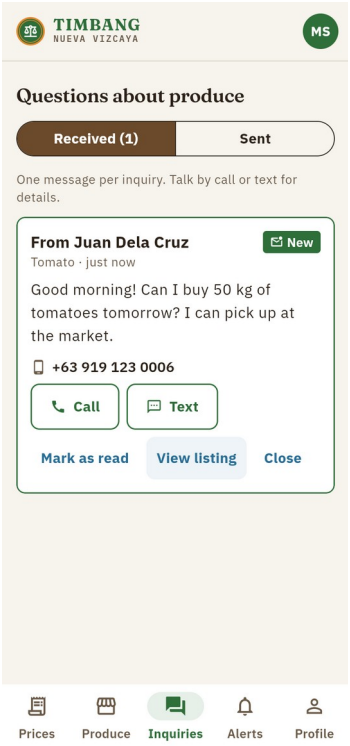


Figure E-16. Inquiry inbox

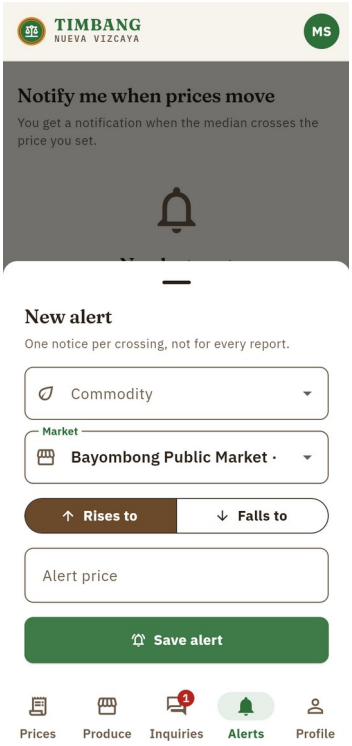


Figure E-17. Creating a price alert

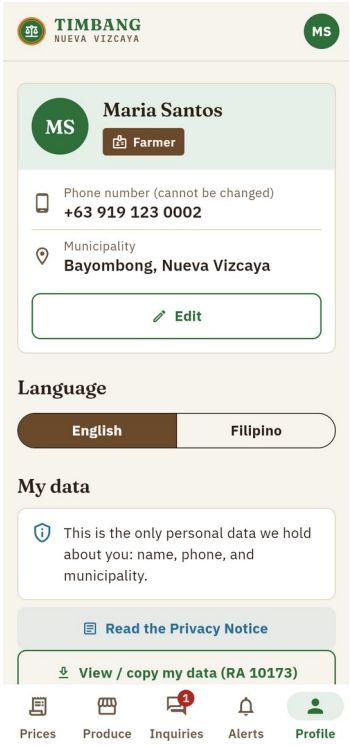


Figure E-18. Profile and my data

5. Administrator dashboard

Administrators open the dashboard from Profile (**Open the admin dashboard**) on the phone, or directly after logging in on the website. The dashboard has these pages:

- **Overview:** counts of active vegetables and markets, demo items, and the pilot metrics (median price age, active farmers, listings, inquiries per listing, freshness per market).
- **Commodities and Markets:** add or edit items one at a time, or import the office's Excel or CSV file as it is. Items are never deleted, only set inactive.
- **Prices:** import official prices from a file (one row per price, or one column per market), add a single official price, and monitor incoming reports.
- **Users:** approve price reporters, assign their markets, change roles, suspend accounts.
- **Moderation:** review user reports and prices far from the median.
- **SMS prices:** try the SMS price query exactly as a basic-phone user would.
- **Audit log:** a record of every sensitive administrator action.

Import order for official data: first the markets, then the vegetables (commodities), then the prices. Every row is checked before anything is saved; rows with errors are listed with their line numbers.

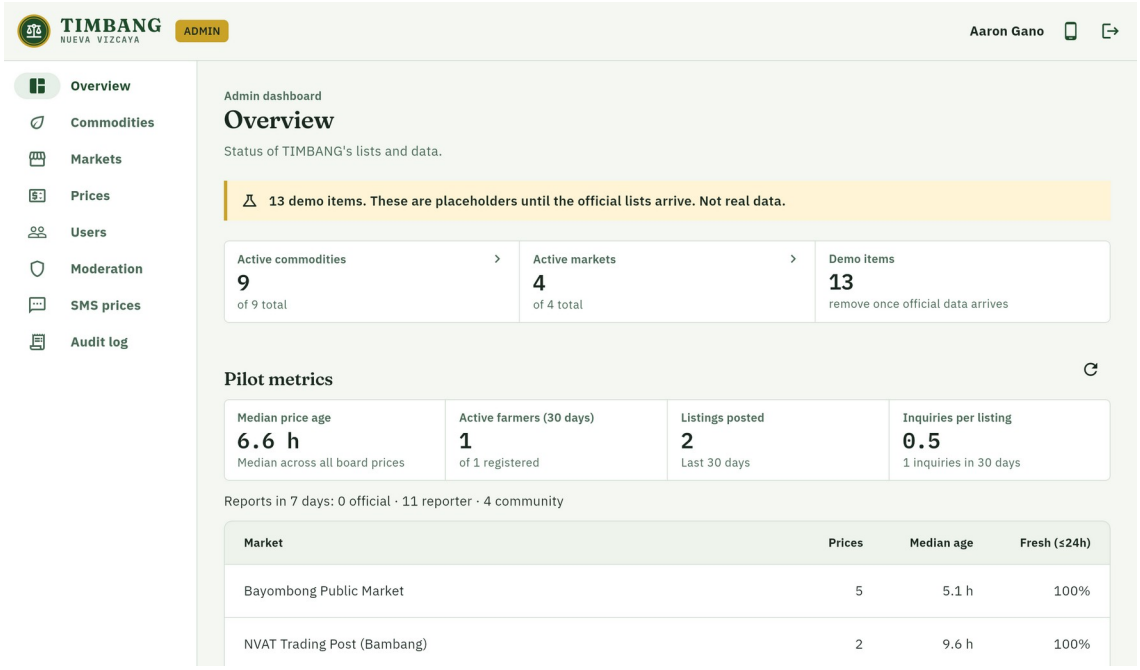


Figure E-19. Administrator overview and pilot metrics

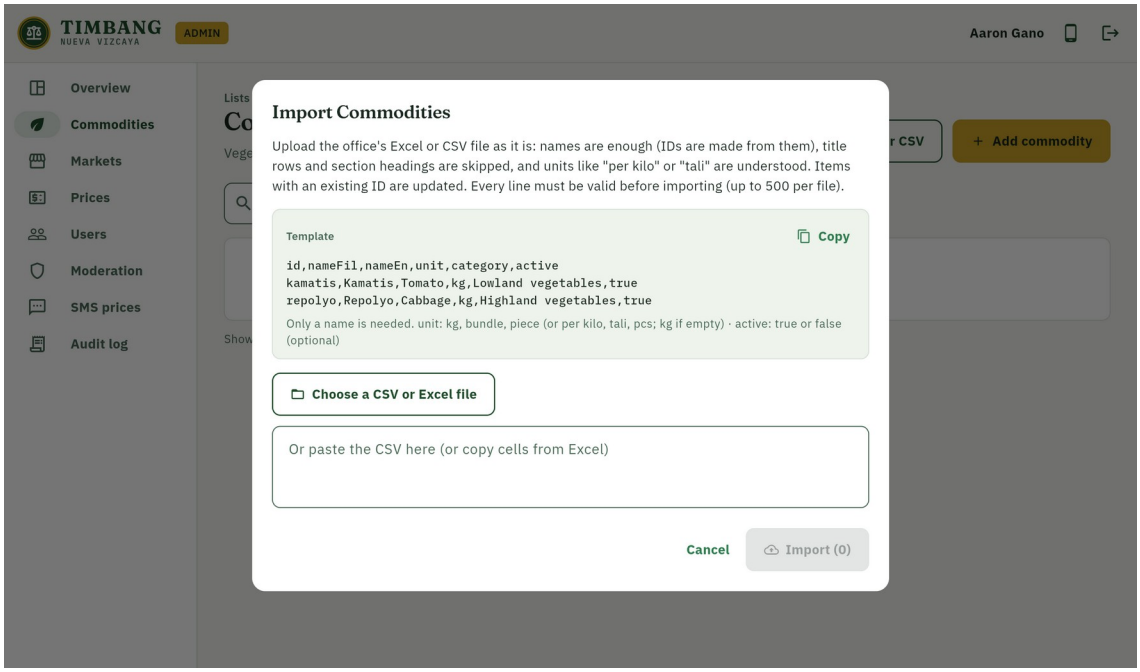


Figure E-20. Importing the vegetable list from Excel or CSV

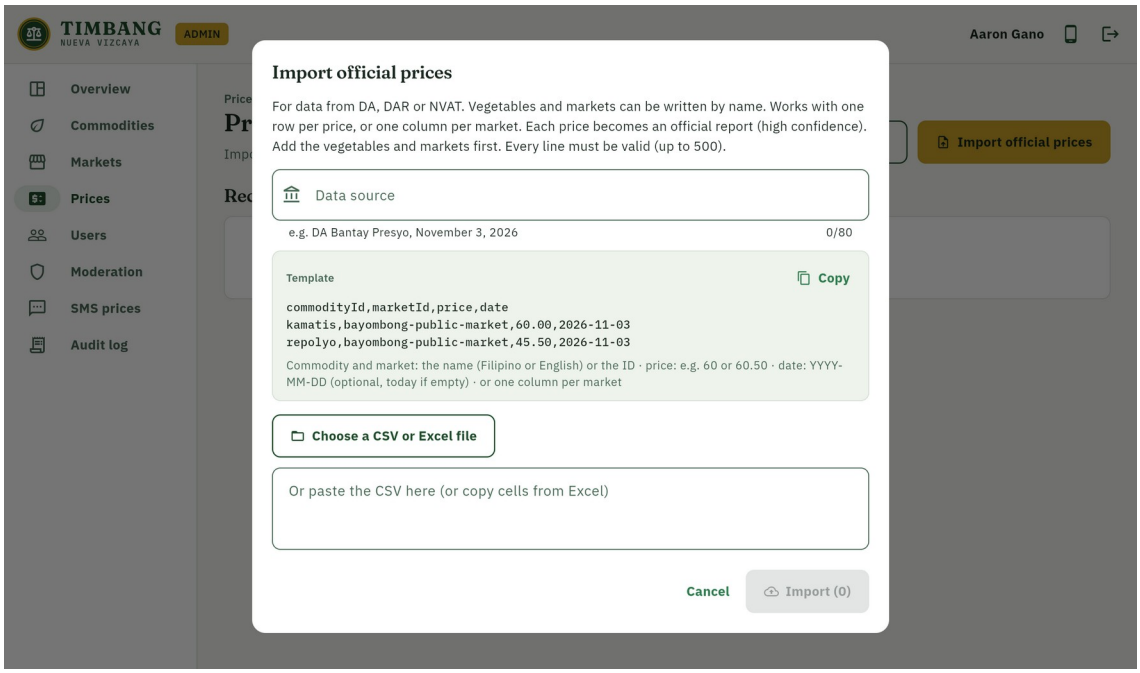


Figure E-21. Importing official prices

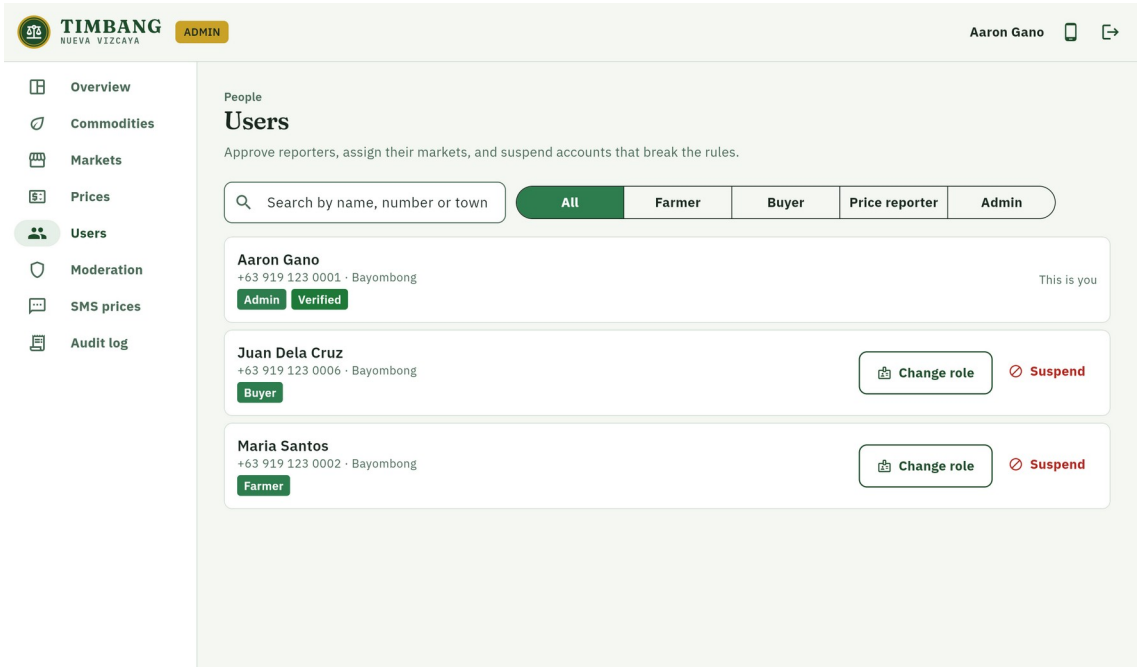


Figure E-22. User and role management

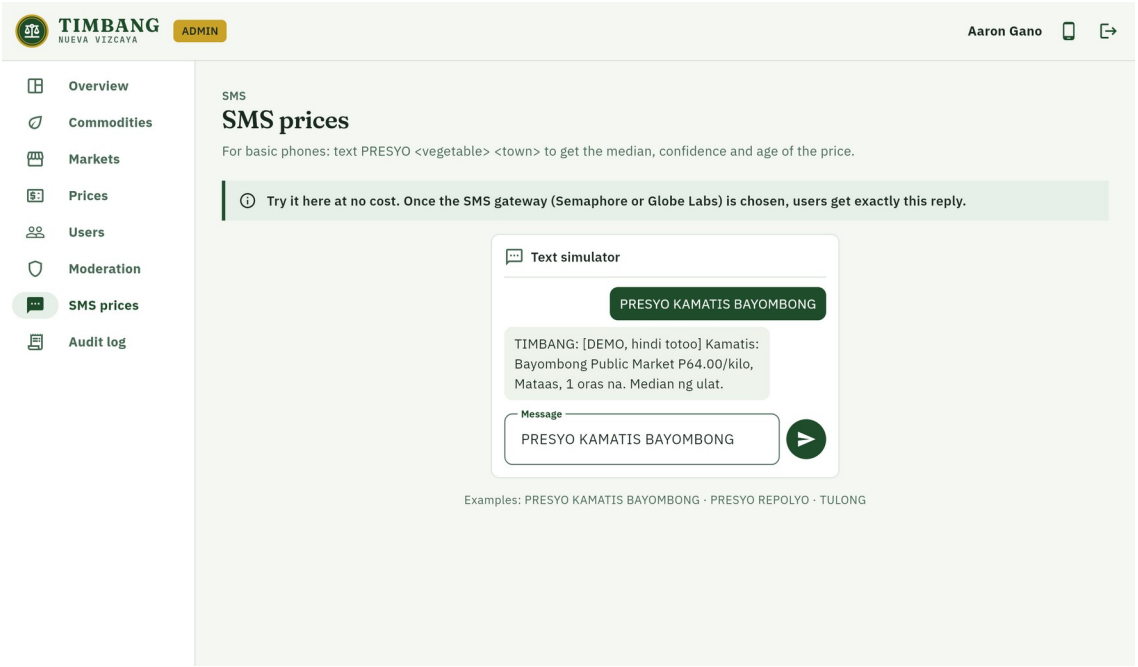


Figure E-23. SMS price query simulator

6. Website

The public website (timbang-nv.web.app) explains TIMBANG and its confidence labels, and lets users log in to the same features as the app from a browser.

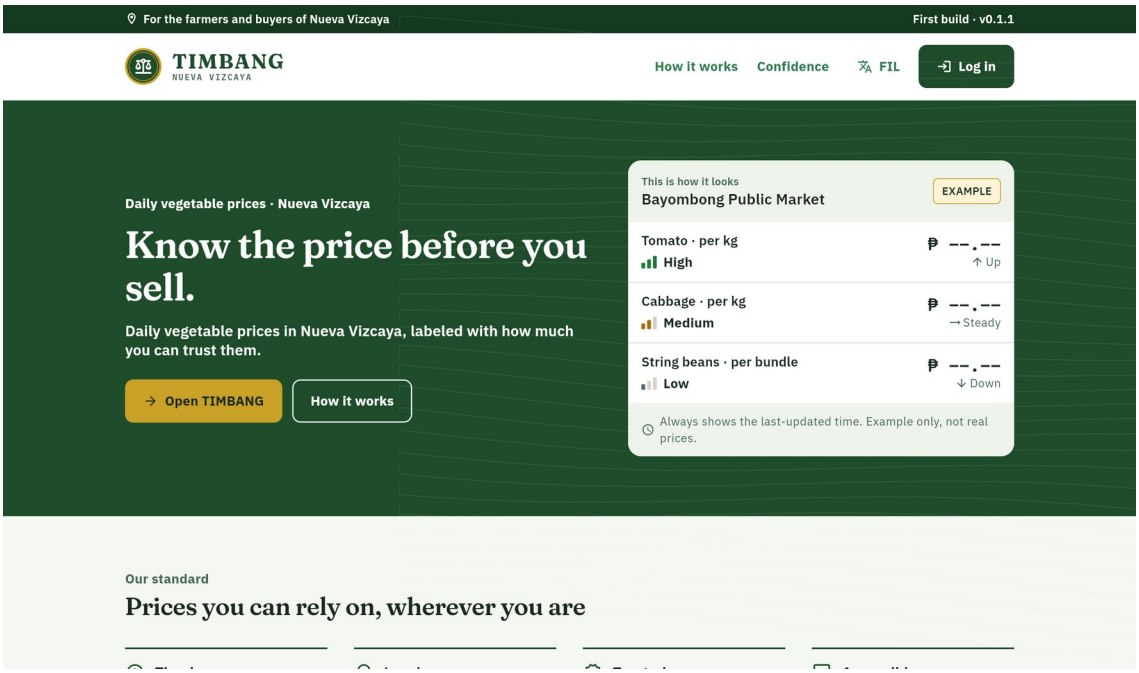


Figure E-24. Public website

E.2 Technical Documentation

1. System overview

TIMBANG consists of a Flutter application built for Android and the web from one code base, a set of Cloud Functions that hold all trusted logic, a Cloud Firestore database protected by security rules, Cloud Storage for listing photos, and an SMS interface for basic phones. Prices and roles are written only by the server; the application can only read them.

Component	Technology	Responsibility
Mobile app	Flutter (Dart), Material 3, Riverpod, go_router	Price board, listings, inquiries, alerts, profile; offline cache
Website and admin dashboard	Flutter web, Firebase Hosting	Public landing page, same user features, administrator dashboard
Authentication	Firebase Authentication (phone OTP), custom claims	Login by SMS code; roles stored as verified claims
Database	Cloud Firestore (asia-southeast1), offline persistence	Users, lists, price reports, price board, listings, inquiries, alerts, audit log
Server logic	Cloud Functions for Firebase (TypeScript, Node.js 22)	Price computation, imports, roles, inquiries, alerts, SMS, data export and erasure
Files	Cloud Storage	Compressed listing photos (max 1 MB, images only)
Notifications	Firebase Cloud Messaging	Inquiry and price-alert notifications
Integrity	Firebase App Check	Blocks requests that do not come from the genuine app (switchable)
SMS	SmsGateway interface (mock; Semaphore or Globe Labs later)	PRESYO <vegetable> <town> price query

2. Data model

Collection	Contents	Who may write
users	Name, phone, municipality, role (display only), verified, status, consent time	Owner (limited fields); server
commodities, markets	Official lists of vegetables and markets	Server (admin functions) only
price_reports	Each submitted or imported price with its source tier and status	Server only
price_board	Computed median, confidence, trend and age per vegetable and market	Server only
listings	Produce posted by farmers; 14-day expiry	Owner farmer (validated)
inquiries	Buyer messages to farmers	Server creates; farmer and buyer update status
alerts	User price alerts	Owner (validated)
abuse_reports	User reports of wrong prices or listings	Any user creates; admin resolves
audit_logs	Every sensitive administrator action	Server only

3. Price confidence method

1. Take all reports for one vegetable at one market from the last 24 hours, excluding reports rejected by an administrator.
2. Compute the median price, never the mean or the latest single report.
3. Set aside reports more than 30% from the median as outliers and recompute the median. Official reports are never set aside; if every report would be an outlier, none are set aside.
4. Assign confidence: High for an official price or three or more reports within 15% of the median including a verified reporter; Medium for verified reporter reports; Low for community reports only.
5. Compute the trend against the previous 24-hour median; a change within ±3% is shown as steady.
6. Keep showing a price with no new report in 24 hours, marked stale with its age.
7. Province view: the median of the market medians, with the range across markets and the lowest confidence among those markets.

4. Security and privacy measures

- Deny-by-default Firestore and Storage rules, with a test for every collection.
- Roles in verified custom claims, set only by an admin-only function; role changes are audited.
- All prices, imports and role changes are performed by Cloud Functions that re-validate input.
- Rate limits: 20 price reports per user per hour, 10 inquiries per hour, 10 SMS replies per number per hour.
- SMS webhook authenticated with HMAC-SHA256 in constant time; phone numbers hashed or masked in logs.
- Firebase App Check support, HTTPS only, and security headers on the website.
- Data Privacy Act of 2012 (RA 10173): only name, phone and municipality are collected; consent is recorded; the full privacy notice must be read before agreeing; users can view, copy and delete their data.
- Threat model (STRIDE) and OWASP MASVS checklist in the project documentation.

5. Testing results

All automated tests were run on November 11, 2026 against the Firebase Local Emulator Suite. All tests passed.

Test suite	What it checks	Tests	Result
Application (flutter_test)	Screens, price board, imports, accessibility (text contrast, touch target size, labels), language	106	All passed

Test suite	What it checks	Tests	Result
Cloud Functions (Jest)	Price confidence, roles, reports, imports, listings, inquiries, alerts, SMS, account deletion	166	All passed
Security rules (rules-unit-testing)	Allowed and denied reads and writes for every collection and for Storage	103	All passed
Integration (Node test runner)	End-to-end function calls on the emulators: prices, roles, catalog, inquiries, SMS, admin	18	All passed
Total		**393**	**All passed**

6. System requirements

User	Requirement
Mobile app	Android 7.0 or newer; mobile data or Wi-Fi to log in and update (the price board stays readable offline)
Website	A current browser (Chrome, Edge, Firefox, Safari)
SMS	Any mobile phone that can send and receive text messages
Administrator	Phone or computer with a browser; official lists as Excel (.xlsx) or CSV files

7. Build and deployment

1. Create the Firebase project (Blaze plan), enable phone sign-in, Firestore (asia-southeast1) and Storage.
2. Connect the app with flutterfire configure and register the Android signing fingerprints (SHA-1, SHA-256).
3. Deploy rules, indexes and functions: firebase deploy --only firestore,storage,functions.
4. Build the Android app: flutter build apk --release -t lib/main_app.dart.
5. Build and publish the website: flutter build web --release -t lib/main_web.dart, then firebase deploy --only hosting.
6. Grant the first administrator role with npm run seed:admin, then import the official lists from the dashboard.

8. Limitations of the first build

- The demonstration lists and prices are placeholders until the official lists from the DA, DAR and NVAT are imported.
- The SMS gateway (Semaphore or Globe Labs) is not yet chosen; SMS works through the mock gateway and the simulator.
- Release builds are signed with a development key; a dedicated release key is needed before wider distribution.
- TIMBANG is an independent student project and is not yet affiliated with or endorsed by the Provincial Government of Nueva Vizcaya, DA, DAR or NVAT.

E.3 Photo Documentation

The following photos document the conduct of the study.

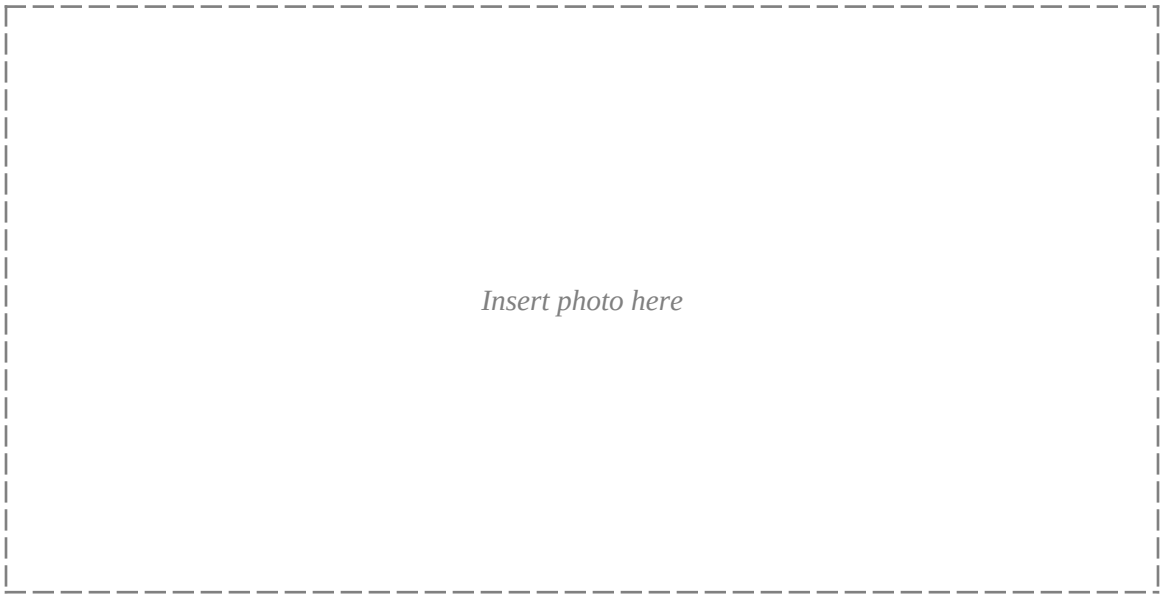


Photo E-1. Consultation with the Office of the Provincial Agriculturist / Office of the Governor
Date: _____ Place: _____

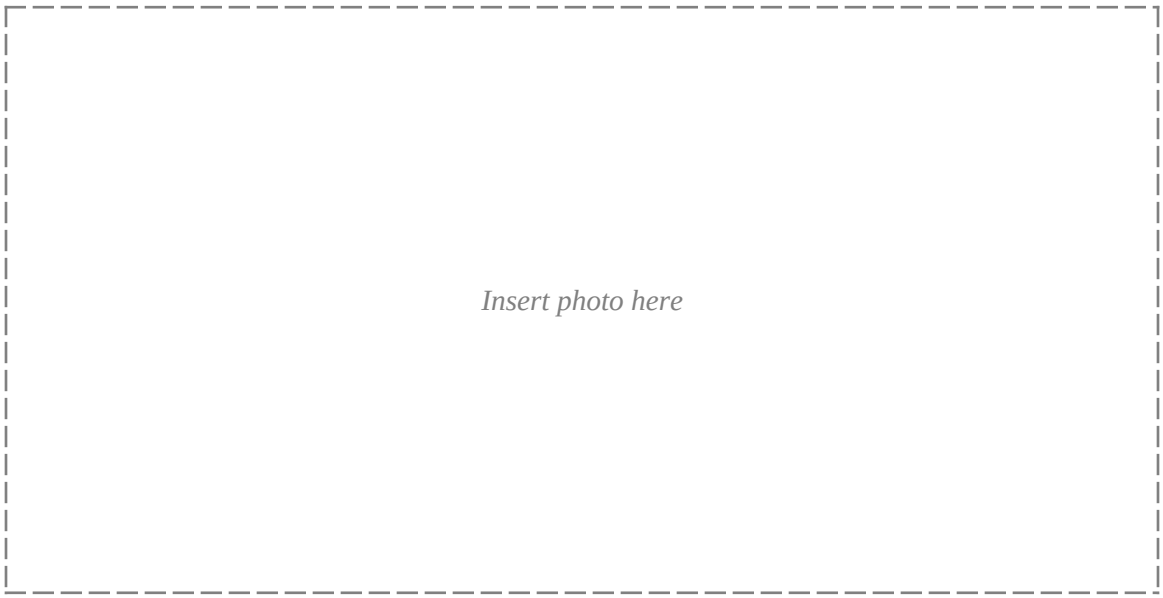


Photo E-2. Data gathering: collection of the official vegetable price lists
Date: _____ Place: _____

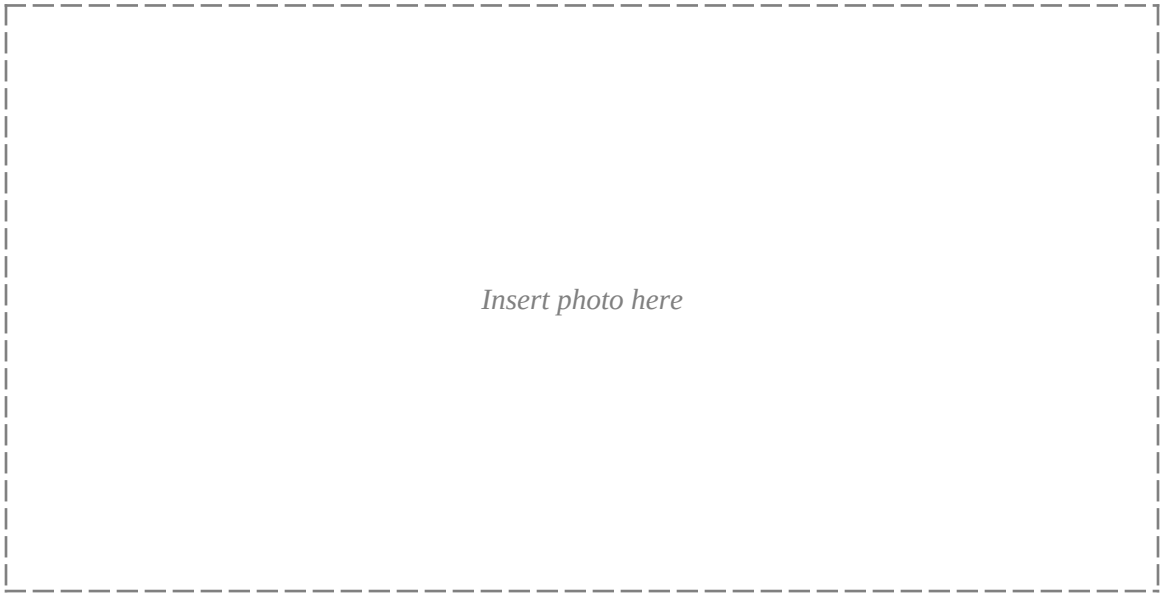


Photo E-3. System testing with farmers

Date: _____ Place: _____

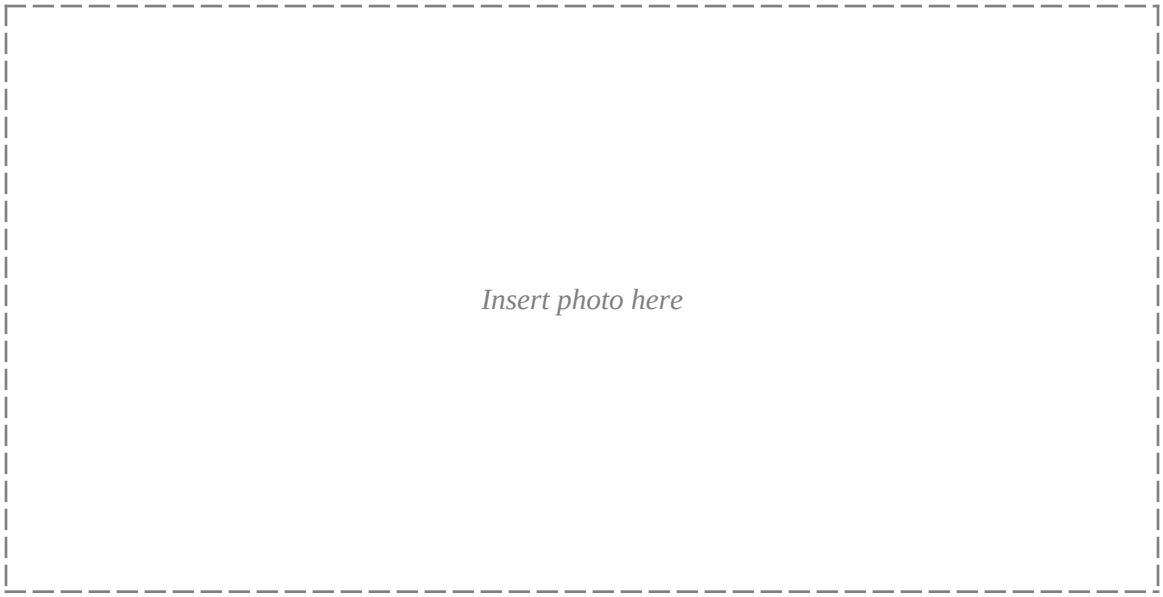


Photo E-4. System testing with buyers

Date: _____ Place: _____

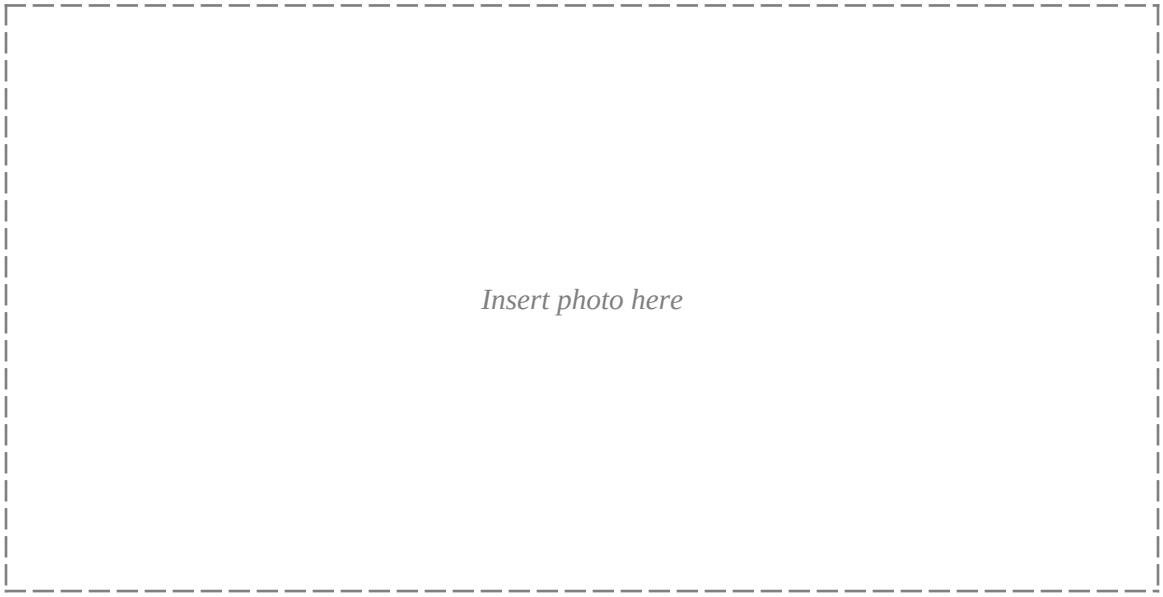


Photo E-5. Usability assessment (System Usability Scale) with respondents

Date: _____ Place: _____

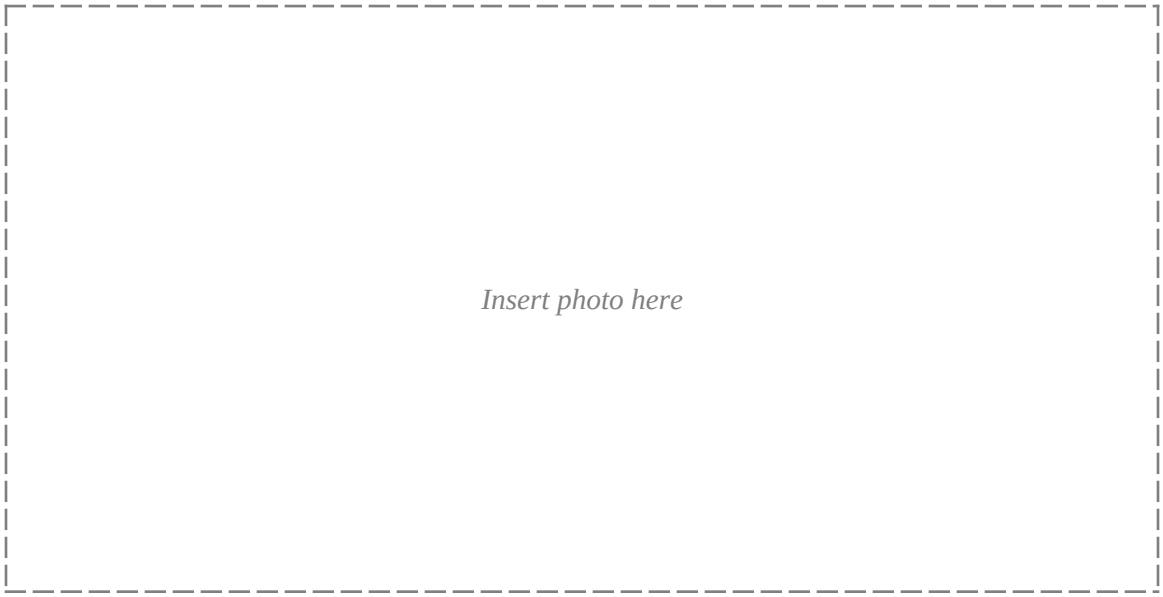


Photo E-6. Presentation of the system

Date: _____ Place: _____